

Looped nanochat: Two-Pass Routing Transformer vs. Baseline — Experiment Report

A controlled comparison of a **two-pass, shared-weight routing-loop Transformer** ("looped") against the stock **nanochat** architecture ("baseline"), trained under identical data, token budget, and schedule at the **d24** (24-layer, ~1.38 B-param) scale on a 4×H100 node.

TL;DR

Under a matched training budget, the looped model **wins or ties every evaluation**:

Metric	Baseline	Looped	Δ
Base CORE (pretraining)	0.2552	0.2783	+9.0% rel
ChatCORE (post-SFT, full sets)	0.2253	0.2521	+11.9% rel
ChatCORE (categorical only)	0.3293	0.3697	+12.3% rel
ARC-Easy	62.04%	68.01%	+5.97 pp
ARC-Challenge	50.00%	52.22%	+2.22 pp
MMLU	37.06%	37.96%	+0.90 pp
GSM8K	2.88%	4.17%	+1.29 pp
HumanEval	10.98%	10.98%	±0.00 pp

The gain costs ~2.5× the training compute (see Cost & speed), most of which is inherent to running the trunk twice, partially offset by a FlexAttention kernel fix that recovered ~2.3× on the routing attention path.

The architecture: two-pass causal routing loop

Baseline nanochat is a standard causal Transformer. The looped variant (`nanochat/gpt.py` , `routing_loop=True`) runs the trunk **twice** per forward:

- Pass 1** runs the ordinary causal Transformer and captures the normalized, RoPE-applied Q/K features from selected deeper layers.
- Pass 2** re-runs the trunk from the token embeddings. Each pass-2 layer receives a **causal attention-logit prior** from one selected pass-1 layer:

$$\text{logits} = q_2 \cdot k_2^T / \sqrt{D} + g \cdot q_1 \cdot k_1^T / \sqrt{D}$$

where `g = softplus(gate_logits)` is a per-layer learned routing gate, and the pass-1→pass-2 wiring is set by `routing_pattern` (default `progressive`).

The additive bias is realized **without materializing an N×N attention matrix**: concatenating `q_aug = [q2, q1]` , `k_aug = [k2, g·k1]` and running ordinary scaled-dot-product attention at width `2D` with softmax scale `1/√D` reproduces the formula exactly. Everything is causal and differentiable; pass-2's LM loss backprops into pass-1 (unless `--routing-detach`), plus a small pass-1 auxiliary LM loss (`first_pass_loss_weight=0.1`).

Two learned scalars are added over baseline: `routing_gate_logits` (one per layer) and `pass_embeddings` (a $2 \times n_{\text{embd}}$ per-pass embedding).

Experimental setup

	Value
Base repo	<code>karpathy/nanochat</code> @ <code>92d63d4</code>

	Value
Model	d24 — 24 layers, <code>n_embd=1536</code> , 12 heads, ~1.38 B params
Hardware	4× H100 80GB (single node)
Pretraining	<code>--depth=24 --target-param-data-ratio=8 --fp8</code> , 5,568 steps, 1,048,576-token batch, ~5.84 B tokens
SFT	SmolTalk + MMLU(×3) + GSM8K(×4), 789,759 rows, 467 steps
Precision	bf16 compute, fp8 matmuls, FlashAttention-3

Both runs are identical except `--routing-loop` (+ `--routing-pattern=progressive --routing-gate-init=0.05`). Gradient accumulation auto-adjusts for the 4-GPU world size (grad-accum 8 baseline / 16 looped at device-batch 16/8), so the optimization trajectory (total batch, tokens, schedule) is matched.

Results in detail

Pretraining

- Baseline final train loss $\approx$ **2.34**; looped $\approx$ **2.59**. (Loss is not directly comparable — the looped pass-2 loss is a different objective.)
- Base **CORE** (22-task aggregate, full sets): baseline **0.2552**, looped **0.2783** — the looped model is a materially better base model.

Post-SFT (chat) — all on full test sets

- Categorical (full): ARC-Easy **68.0** vs 62.0, ARC-Challenge **52.2** vs 50.0, MMLU **38.0** vs 37.1.
- Generative (full): GSM8K **4.17%** vs 2.88% (55/1319 vs 38/1319); HumanEval **10.98%** vs 10.98% (18/164 both).
- **ChatCORE (full, recomputed with the same centered-mean formula)**: looped **0.2521** vs baseline **0.2253**.

Note on ChatCORE: nanochat's in-loop ChatCORE during SFT samples only 24 problems per generative task, which is too noisy for GSM8K/HumanEval (it reported GSM8K 12.5% vs the true full-set 2.88% for baseline). The ChatCORE figures above are recomputed from **full**-set GSM8K/HumanEval and full categorical sets for a fair comparison.

Cost & speed

Phase	Baseline	Looped
FLOPs / token	3.96e8	1.19e9 (~3×)
Pretraining throughput	~495 K tok/s	~208 K tok/s
Pretraining wall-clock (4×H100)	~3.3 h	~7.8 h
Approx. training cost @ \$16.36/hr	~\$60	~\$130

The looped forward is intrinsically ~3× the matmul FLOPs (two passes, pass-2 at doubled Q/K width). Naively it ran ~5.8× slower than baseline because the width-2D routing attention fell off the fused FlashAttention-3 kernel onto generic SDPA. Routing the training attention through `torch.compile d FlexAttention` (see Engineering fixes) recovered ~2.3× (82 K → 208 K tok/s), bringing the real slowdown to ~2.4×.

Inference / decode

Looped decode is **latency-bound**: ~30 ms/step regardless of batch size, scaling linearly to **4160 tok/s at batch 128**. Single-stream is ~34 tok/s ($\approx$ 2× baseline's 73 tok/s — the two passes), which is fine for interactive chat. The decode KV-cache path deliberately keeps SDPA (correctness-sensitive, T=1).

Engineering fixes made during this work

All in `nanochat/` :

1. **FlexAttention routing path** (`flash_attention.py` , `gpt.py`) — the width- 2D SDPA routing attention is $\sim 2.6\times$ slower than baseline attention on H100 because it can't use FA3. Fold the gate into the routing keys (`k_aug = [k, g.k1]`) and run `torch.compile` d FlexAttention (fused even at the wider head dim). Validated equivalent to the additive-bias reference (max-abs err ~ 0.008 , full and windowed); **2.3 $\times$ training throughput**. Compiled with `dynamic=True` so eval's varying sequence lengths don't trigger per-shape recompiles that fall back to the unfused (full-scores) kernel.
2. **Distributed-optimizer fix** (`optim.py`) — `DistAdamW`'s `reduce_scatter` requires each parameter's leading dim divisible by `world_size` ; the routing `pass_embeddings` (`[2, n_embd]`) violates this on 4 GPUs. Fall back to `all_reduce` for such parameters. Without this the looped run crashes at optimizer step 1 on any multi-GPU world where `world_size` $\nmid 2$.
3. **SFT checkpoint config** (`chat_sft.py`) — the SFT save reconstructed `model_config` field-by-field and dropped the routing fields, so a looped SFT checkpoint would silently rebuild as a **non-routing** model. Now persists `routing_loop` , `routing_pattern` , `routing_gate_init` , `routing_detach` , `first_pass_loss_weight` .
4. **Batched generative eval** (`engine.py` , `chat_eval.py`) — `Engine.generate_prompts` generates a batch of distinct prompts at once (decode is latency-bound, so this is near-free), with the tool-use (calculator) state machine handled per row. Validated to give the same accuracy as the one-at-a-time path. Uses exact-length bucketing for provable correctness; a padded/masked variant for larger real-world speedups on variable-length prompts is left as follow-up.

Test-time pass scaling (negative result)

Since the architecture is a loop, a natural question is whether running **more than 2 passes at inference** — with no retraining — buys extra quality (a "free" test-time-compute knob). We generalized the forward to `K` iterations (`GPT.forward_iterated` , inference-only: pass 0 raw + captures; each later pass routes from the previous pass's Q/K and re-captures) and measured val bpb over 21 M tokens (4 $\times$ H100) as a function of `K` :

K passes	val bpb ($\downarrow$)
1 (pass-0 only)	0.72608
2 (trained config)	0.71092
3	0.72007
4	0.72214
5	0.72386
6	0.72451
8	0.72497

Inference-only pass scaling does not work here. $K=2$ (exactly what the model was trained with) is optimal; every additional pass monotonically increases bpb, asymptoting just past the untrained $K=1$ level. This is the expected consequence of the model never being trained to consume the routing prior recursively — extra passes re-inject a signal the network can't iterate on, and it drifts. Getting real test-time scaling out of the loop would require training with a variable/larger number of passes (e.g. randomizing K), which is a training change, not a free inference lever. Reproduce with [scripts/tts_pass_scaling.py](#) .

Reproduce

```
# baseline
bash runs/speedrun_baseline.sh          # tag baseline-d24
# looped
bash runs/run_looped.sh                 # tag looped-d24
# full generative eval (either tag)
```

```
torchrun --standalone --nproc_per_node=4 -m scripts.chat_eval -- \
-i sft -g looped-d24 -a "GSM8K|HumanEval"
```

Takeaways

- At d24 / ~\$100-tier scale, the two-pass routing loop is a **real quality win** (+9% base CORE, +12% ChatCORE) for ~2.4× training compute.
- The biggest generative gain is on **GSM8K** (+45% relative), consistent with the routing prior helping multi-step reasoning; HumanEval is unchanged at this scale.
- The kernel choice matters enormously: expressing the routing bias so it stays on a fused attention kernel (FlexAttention) is the difference between ~2.4× and ~5.8× training slowdown.

Models and full artifacts: see the HuggingFace model cards for `baseline-d24` and `looped-d24` .